

Week 5: Quarto & Plotting

September 23, 2026

WHY QUARTO: LITERATE PROGRAMMING

*Treat programs as a “literature” understandable to **human beings**.*
— Donald Knuth

An analysis is more than its code. Someone (a teammate, a reviewer, you in three months) needs to know *why* each step happened and what the results mean. A Quarto document (`.qmd`) weaves **prose, code, and the code’s output** into one file, then renders it to a finished HTML page, PDF, or Word doc. Change the data, re-render, and every number and chart updates — nothing is copy-pasted by hand.

Every project deliverable this semester is a `.qmd`, so this is the format your work gets graded in. To try it: open `quarto_demo.qmd` in Positron and click **Preview**.

ANATOMY OF A .QMD FILE

A `.qmd` has three parts: a **YAML header** (the settings), **markdown text**, and **R code**.

1. The header

The header sits between two `---` lines at the very top of the file:

```
---
title: Your title
author: Author name
toc: true
format:
  html:
    theme: united
---
```

Setting	Effect
<code>format: html</code>	Web page (the default for our work)
<code>format: pdf</code>	PDF via LaTeX
<code>format: docx</code>	Microsoft Word
<code>toc: true</code>	Adds a table of contents
<code>theme: united</code>	Changes the HTML look

No LaTeX for PDF output? Run `tinytex::install_tinytex()` once in R.

2. Markdown text

Bookmark commonmark.org/help, and when you have 10 minutes, do its [tutorial](#).

Type this	To get
<code>#</code> Header 1 (up to six)	Section headers
<code>*italic* or <code>_italic_</code></code>	<i>italic</i>
<code>**bold**</code>	bold
<code>~~strikethrough~~</code>	strikethrough
backticks around text	code text
a dash and a space	bulleted list item
1. and a space	numbered list item
[Download R](http://r-project.org)	Download R

3. R code

Inline code runs inside a sentence: writing ``r 3 + 4`` in the text prints 7 in the rendered document.

Code chunks start with three backticks and `{r}` on their own line and end with three backticks. Positron’s **Insert Code Cell** does this for you.

Chunk options go at the top of the chunk, each on a line starting with a hash and a pipe:

Option	Shows code?	Shows output?	Runs?
(default)	yes	yes	yes
<code>echo: false</code>	no	yes	yes
<code>eval: false</code>	yes	no	no
<code>include: false</code>	no	no	yes

Also handy: `warning: false, message: false, fig-width, fig-height`. All options are in the [Quarto cell reference](#).

The setup chunk

Make the first chunk a `setup` chunk with `include: false`. It sets global options, loads libraries, and reads the data, so every later chunk can use them:

```
#| label: setup
#| include: false

knitr::opts_chunk$set(
  warning = FALSE,
  message = FALSE,
  fig.width = 7.252,
  fig.height = 4
)

library(tidyverse)
bears <- read_csv(file.path('data',
  'bear_killings.csv'))
```

A rendered `.qmd` starts from a **clean R session**. An object that exists only in your console doesn’t exist when you render. If the document needs it, create it in a chunk.

Putting it together

A complete, renderable `.qmd`, in the shape of the `bears.qmd` practice:

```
---
title: "Bears Analysis"
format: html
---

``{r}
#| label: setup
#| include: false
library(tidyverse)
bears <- read_csv(file.path('data',
  'bear_killings.csv'))
```

Which months have the most bear killings?

``{r}
bears %>%
 count(month) %>%
 arrange(desc(n))
``
```

`count(variable)` counts the rows for each unique value, the quickest way to answer “which is more common?” Add `arrange(desc(n))` to put the largest first.

## PLOTTING WITH GG PLOT2

`ggplot2` (Hadley Wickham, 2005) implements Leland Wilkinson’s *Grammar of Graphics*: a chart is built from **layers**, added one at a time with `+`.

| Layer         | Function               | Question it answers       |
|---------------|------------------------|---------------------------|
| 1. Data       | <code>ggplot()</code>  | What data?                |
| 2. Aesthetics | <code>aes()</code>     | Which variables go where? |
| 3. Geometries | <code>geom_*()</code>  | Points? Bars? Lines?      |
| 4. Labels     | <code>labs()</code>    | What do the axes say?     |
| 5. Theme      | <code>theme_*()</code> | How does it look?         |

### Aesthetic Controls

|                    |                                 |
|--------------------|---------------------------------|
| <code>x, y</code>  | Position on the axes            |
| <code>color</code> | Points, lines, and bar outlines |
| <code>fill</code>  | Inside of bars and areas        |
| <code>shape</code> | Point symbol (max 6 groups)     |
| <code>size</code>  | Point size or line width        |
| <code>alpha</code> | Transparency, 0 to 1            |

Map any of these aesthetics to a variable inside `aes()`, or set it to a constant outside `aes()`. A full chart, all five layers:

```
mpg %>%
 ggplot(aes(x = displ, y = hwy)) +
 geom_point() +
 labs(
 x = "Engine displacement (liters)",
 y = "Highway fuel economy (mpg)",
 title = "Larger engines are less fuel efficient"
) +
 theme_bw()
```

Note the two different joiners: the pipe passes data *into* `ggplot()`; after that, layers are joined with a plus sign.

### Mapping vs. setting

**Inside `aes()` you map a variable; outside `aes()` you set a constant.** `geom_point(aes(color = class))` colors each point by its vehicle class. `geom_point(color = 'blue')` makes every point blue. Put a fixed value inside `aes()` and `ggplot` treats it as a one-level variable, not a color.

**Where to put `aes()`.** In `ggplot(aes(...))` the mapping applies to every layer. Inside one geom, say `geom_point(aes(...))`, it applies to that layer only, so a following `geom_line()` with no mapping errors out.

### The main geoms

| Geom                       | Makes                        | Notes                                      |
|----------------------------|------------------------------|--------------------------------------------|
| <code>geom_point()</code>  | Scatterplot                  | Map color or shape                         |
| <code>geom_line()</code>   | Connects dots, left to right | Messy on scattered data                    |
| <code>geom_smooth()</code> | Best-fit trend line          | <code>se = FALSE</code> drops the band     |
| <code>geom_col()</code>    | Bars of a given height       | Summarize first, e.g. <code>count()</code> |

**Points and lines use `color`; bars use `fill`.** `color` on a bar only outlines it.

### Bar chart recipe

```
mpg %>%
 count(class, drv) %>%
 ggplot() +
 geom_col(
 aes(x = reorder(class, n), y = n, fill = drv),
 position = "dodge", # side-by-side, not stacked
 width = 0.7
)
```

- `count(class)` makes the summary data frame; a `fill` variable must be counted too, as in `count(class, drv)`.
- `reorder(class, n)` sorts bars by count instead of alphabetically.
- For many long category names, put them on the y axis to keep them readable.

### Facets

Too many colors to tell apart? Split the chart into small multiples:

| Code                               | Result                                                 |
|------------------------------------|--------------------------------------------------------|
| <code>facet_wrap(~class)</code>    | One panel per class                                    |
| <code>facet_grid(drv ~ cyl)</code> | Rows by <code>drv</code> , columns by <code>cyl</code> |

### Themes

Built in: `theme_bw()`, `theme_minimal()`, `theme_classic()`, `theme_void()`. More in the **ggthemes** package (`ggthemes::theme_economist()`) and **hrbrthemes** (`hrbrthemes::theme_ipsum()`).

## YOUR PROJECT: DEFINING ATTRIBUTE LEVELS

Now that your attributes are in the Model Relationships Table, give each one **levels**, the specific values respondents will see.

| Kind       | Example levels                     |
|------------|------------------------------------|
| Continuous | Price: 1, 2, 3, 4, 5 (\$)          |
| Continuous | Power output: 60, 80, 120 (W)      |
| Discrete   | Color: Red, Blue, Yellow           |
| Discrete   | Material: Plastic, Aluminum, Glass |

Look at competitors. Choose values that cover the full range on the market today, and maybe some of the future.

## CONJOINT SURVEY COMPONENTS

Think of the survey as a **conversation**: add transition pages ("Great job! Now we'll ask you about...") between sections.

| Part        | Contents                              | Screen for        |
|-------------|---------------------------------------|-------------------|
| 1. Intro    | Welcome, consent, eligibility         | Target population |
| 2. Conjoint | Education, practice question, choices | Random answers    |
| 3. Other    | Critical info, demographics, finale   | —                 |

- Education pages** explain each attribute before the choices. Relative comparisons help: "1/2 lb, about 1 cup of water".
- The practice question** doubles as an **attention check**.
- After the choices**, you can also drop respondents who picked the same option every time.

## ERRORS YOU WILL HIT THIS WEEK

| Problem                                       | Usual cause                                                   |
|-----------------------------------------------|---------------------------------------------------------------|
| <code>object not found</code> on render       | Created in console, not in a chunk                            |
| <code>could not find function "ggplot"</code> | No <code>library(tidyverse)</code> in setup                   |
| <code>cannot open file 'data/...'</code>      | Folder not opened in Positron                                 |
| Plot doesn't show the geom                    | Plus sign at the start of a line, not the end                 |
| Bars ignore the color                         | Used <code>color</code> ; bars need <code>fill</code>         |
| Every point is one odd color                  | Constant put inside <code>aes()</code>                        |
| <code>missing aesthetics: x</code>            | <code>aes()</code> is only in another geom and <code>y</code> |

## BEFORE NEXT CLASS

- Do [HW 4: Intro to surveydown](#), due **Mon, Sep 28**.
- With your team, write the [Survey Plan](#), due **Tue, Sep 29**.
- Finish `bears.qmd` and the practice charts in `ggplot2.qmd` if you didn't in class.

You should be able to:

- Create a `.qmd` with a header, markdown text, and code chunks, and render it.
- Choose between `echo`, `eval`, and `include` to control what a chunk shows.
- Build a `ggplot` layer by layer, from data to theme.
- Explain the difference between mapping a variable and setting a constant.
- Make sorted, filled bar charts and faceted plots.
- Define levels for your project's attributes and sketch your survey's three parts.